Traffic Control Simulator Design and Test Report

SIT315-Task M3.T3D

1 Introduction

I extended the previous traffic simulator to use MPI processes and producer and consumer threads. The program reads traffic records and prints the top N busiest traffic lights for each hour. Each record contains a timestamp, a traffic light ID and a number of cars.

The tested outputs matched the sequential program. However MPI was slower than the sequential program on this machine. Four MPI processes gave the best MPI times. The stress test did not finish within the chosen 60 second limit.

2 Program design

All MPI ranks access the same input file. Each rank reads a separate byte range. A range boundary is moved past a partial line, while the previous range finishes that line. This avoids reading a record twice or losing part of a record. Empty ranges are allowed. Rank zero also processes data and collects the final results.

Each rank creates the requested number of producers and consumers. Producers read batches of records and put them in a bounded queue. Consumers remove batches and add car counts to their own maps. After the threads finish, the main thread combines these maps. The queue capacity is measured in records.

3 Map and Reduce

Map: each record becomes a key and a value. The key contains the hour and traffic light ID. The value is the car count. Repeated keys are summed locally before MPI communication. This local combining reduces the number of values that must be sent.

Exchange: a stable hash of the hour and light ID assigns each key to an owner rank. The owner is hash(key) modulo the number of ranks. MPI_Alltoall exchanges message sizes. MPI_Alltoallv exchanges the variable length text messages containing partial totals.

Reduce: each owner adds all partial totals for its keys. The reduced keys are then sent to rank zero using another variable length exchange. Rank zero groups results by hour, sorts by car count from highest to lowest and prints the top N lights. Equal totals are ordered by light ID.

Implementation distinction: the MPI consumers update maps of totals. The final ranking is sorted after reduction, rather than maintained as a sorted ranking after every consumed record. The task sheet asks for consumers to update a sorted list. This design difference remains to be checked against that instruction before final submission.

4 Data structures and thread safety

Structure	Purpose	Thread safety
Bounded deque of batches	Stores records waiting for consumers	Mutex and condition variables
Input file stream	Reads the byte range for one rank	File mutex shared by producers
Private consumer maps	Stores totals by hour and light ID	One consumer owns each map
Combined rank map	Stores local totals before exchange	Main thread after worker joins
Message strings and vectors	Stores MPI counts and message buffers	Main thread only
Hourly ranking vectors	Sorts final totals and selects top N	Rank zero main thread

Producers block when the queue is full. Consumers block when it is empty. Condition variables wake waiting threads when space or records become available. Closing the queue lets consumers finish after the remaining batches are removed. Cancellation wakes waiting threads if a worker reports an error.

MPI is started with MPI_THREAD_FUNNELED. Only the main thread calls MPI. The exchange, gather and reduction calls are blocking collectives. Worker threads finish before the data exchange. This design does not overlap MPI communication with worker computation.

5 Sequential solution and build commands

The previous traffic_sim.cpp provides the sequential baseline. It reads the same input, sums car counts and returns the same hourly ranking. The data generator creates repeatable input with 12 measurements per light per hour.

```
mpic++ -std=c++17 -O3 -pthread traffic_mpi.cpp -o traffic_mpi
c++ -std=c++17 -O3 -pthread traffic_sim.cpp -o traffic_sim
c++ -std=c++17 -O3 generate_data.cpp -o generate_data
```

All three compilation commands passed. Figure 5 shows the commands and results.

```
./traffic_sim seq data/traffic_demo.csv 3
mpirun --bind-to none -np 2 ./traffic_mpi data/traffic_demo.csv 3 1 1 1024
```

The MPI arguments after the input file are top N, producers per rank, consumers per rank and queue capacity. The tests used both 1 producer with 1 consumer and 2 producers with 2 consumers. The --bind-to none option lets the threads use available CPUs without restricting each rank to one CPU.

6 Machine specifications

Item	Specification
CPU	Apple M4
Physical cores and logical threads	10 cores and 10 logical threads
RAM	16 GiB
OS and architecture	macOS 26.1, arm64
MPI and compiler	Open MPI 5.0.10; Apple Clang 17.0.0

7 Correctness tests

The hand check file contains seven records across two hours. For 08:00, TL01 has 10 + 25 = 35 cars, TL02 has 30 + 5 = 35 cars and TL03 has 20 cars. TL01 comes before TL02 because their totals are equal. For 09:00, TL03 has 40 cars and TL01 has 7 cars. Figure 1 shows these results.

For the performance datasets, a comparison script checked the printed hourly top three rankings, record counts and hour counts against the sequential output. All 36 runs passed these comparisons. This verifies the reported results; it does not compare every unprinted light total.

8 Performance method and results

Dataset	Lights	Hours	Records
Small	1000	10	120,000
Medium	1000	100	1,200,000
Large	1000	500	6,000,000

Each workload was run sequentially and with 2, 4 and 8 MPI processes. Each configuration was repeated three times. The table uses the median of the three reported execution times. MPI tests used one producer, one consumer and a queue capacity of 1024 records per rank. Top N was 3.

Dataset	MPI ranks	Sequential s	MPI s	Speedup
Small	2	0.044159	0.131646	0.335
Small	4	0.044159	0.112236	0.393
Small	8	0.044159	0.248392	0.178
Medium	2	0.476441	1.314676	0.362
Medium	4	0.476441	1.068991	0.446
Medium	8	0.476441	2.415629	0.197
Large	2	2.598545	6.528751	0.398
Large	4	2.598545	5.387850	0.482
Large	8	2.598545	12.556191	0.207

Speedup = sequential median time / MPI median time. A value below 1 means MPI is slower. Workload size and runtime both increased clearly. Four processes were fastest among the MPI configurations, but none beat the sequential baseline.

These are program reported processing times. MPI startup and final output printing are outside its internal timer. The sequential and MPI programs use different update and ranking methods, so this comparison measures the two implementations rather than an isolated effect of MPI alone.

9 Scaling limits and uneven work

MPI adds queue coordination, message packing, data exchange and final collection. All processes run on one machine and share its storage and memory bandwidth. Eight ranks create 16 worker threads in addition to the main threads, on a CPU with 10 cores. Competition for CPU time is a possible reason for the slower eight rank results. No separate phase timings were collected, so these explanations are possible causes rather than measured bottlenecks.

Byte ranges keep input sizes approximately balanced. Local combining makes a frequent key cheaper to exchange because each rank sends its partial total instead of every original record. Hash ownership spreads keys across ranks, but does not guarantee equal key counts. Variable length messages and zero message counts allow unequal or empty ranks to participate correctly.

The uneven dataset contained 1006 records. Of these, 1001 used TL01 in the same hour. Five other lights each had one record. It passed with 2, 4 and 8 ranks. At eight ranks, input counts were 125 or 126 records, but two ranks owned zero reduced keys. The seven record hand check also passed with eight ranks, including rank 7 with zero input records. Figure 6 shows the results.

Remaining limits include waiting for the slowest rank, uneven owner key counts and collecting all reduced keys at rank zero. Text serialization also adds work. Possible future changes include compact binary messages and sending only each owner's top N candidates per hour after reduction. These changes were not implemented in the tested version.

10 Stress test

The stress dataset used 1000 lights and 5000 hours, giving 60 million records. The run used eight MPI processes, one producer and one consumer per rank, top N of 3 and a queue capacity of 1024 records. A Python wrapper enforced a 60 second limit.

```
mpirun --bind-to none -np 8 ./traffic_mpi data/traffic_stress.csv 3 1 1 1024
```

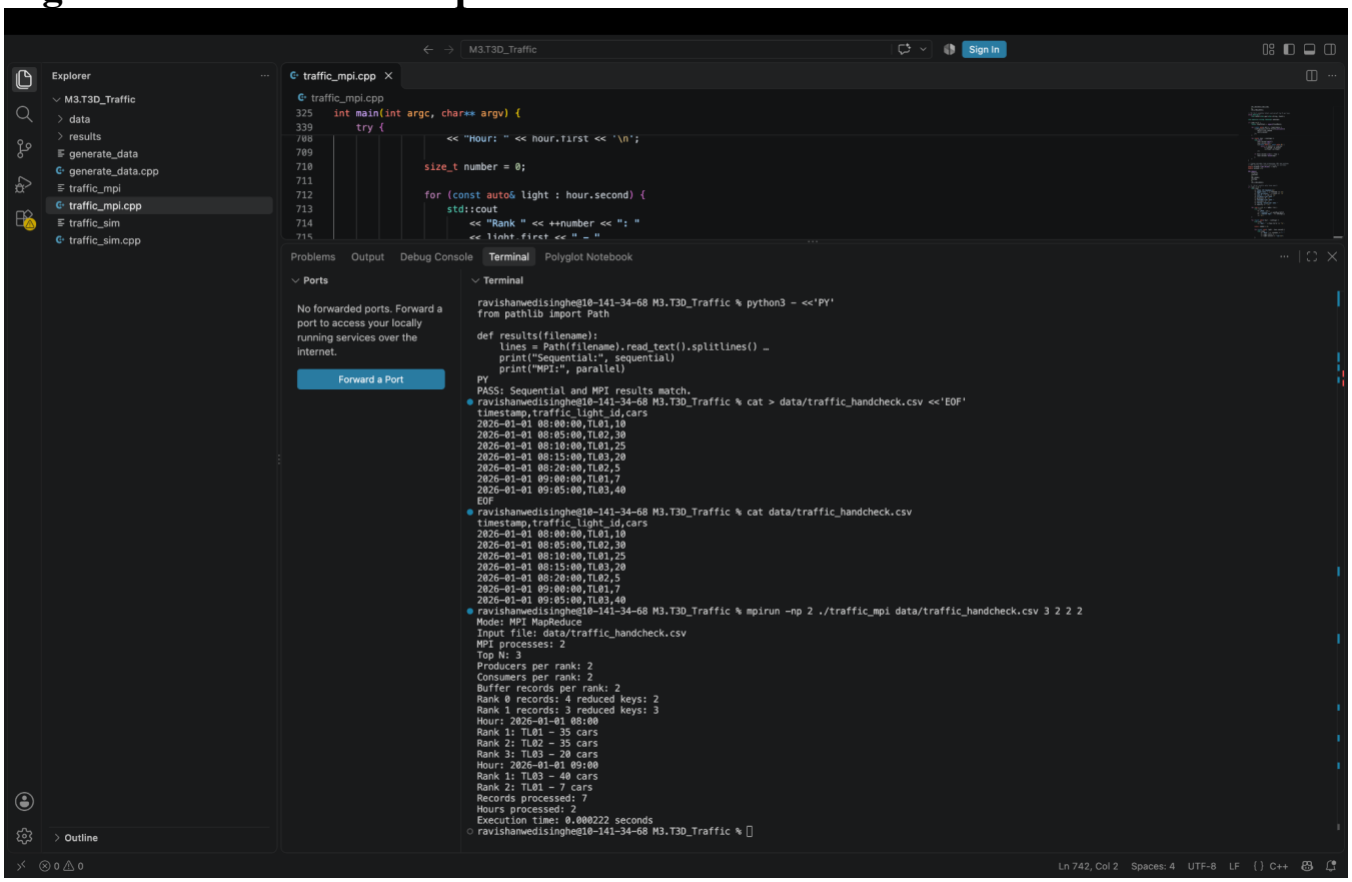
The observed wall time was 60.007 seconds. The wrapper stopped the run because it did not finish within the limit. The launcher exit code was 1 after termination. The reason for DNF was the imposed timeout. The log showed no output when checked. This is not evidence of an out of memory error, crash or deadlock. Stress wall time includes startup and shutdown, unlike the internal benchmark times.

11 Conclusion

The tests show matching hourly output, increasing workloads and correct handling of uneven and empty ranks. MPI was slower than the sequential program on this machine. The design uses local combining to reduce message volume, but communication and final collection still limit scaling. The consumer sorted list instruction needs a final implementation review before claiming complete compliance.

Submit this design report with the final project source code, the example input file and the running evidence. The final source file was not uploaded for an independent review when this report was prepared. The design description follows the implementation developed in this session and the supplied test evidence.

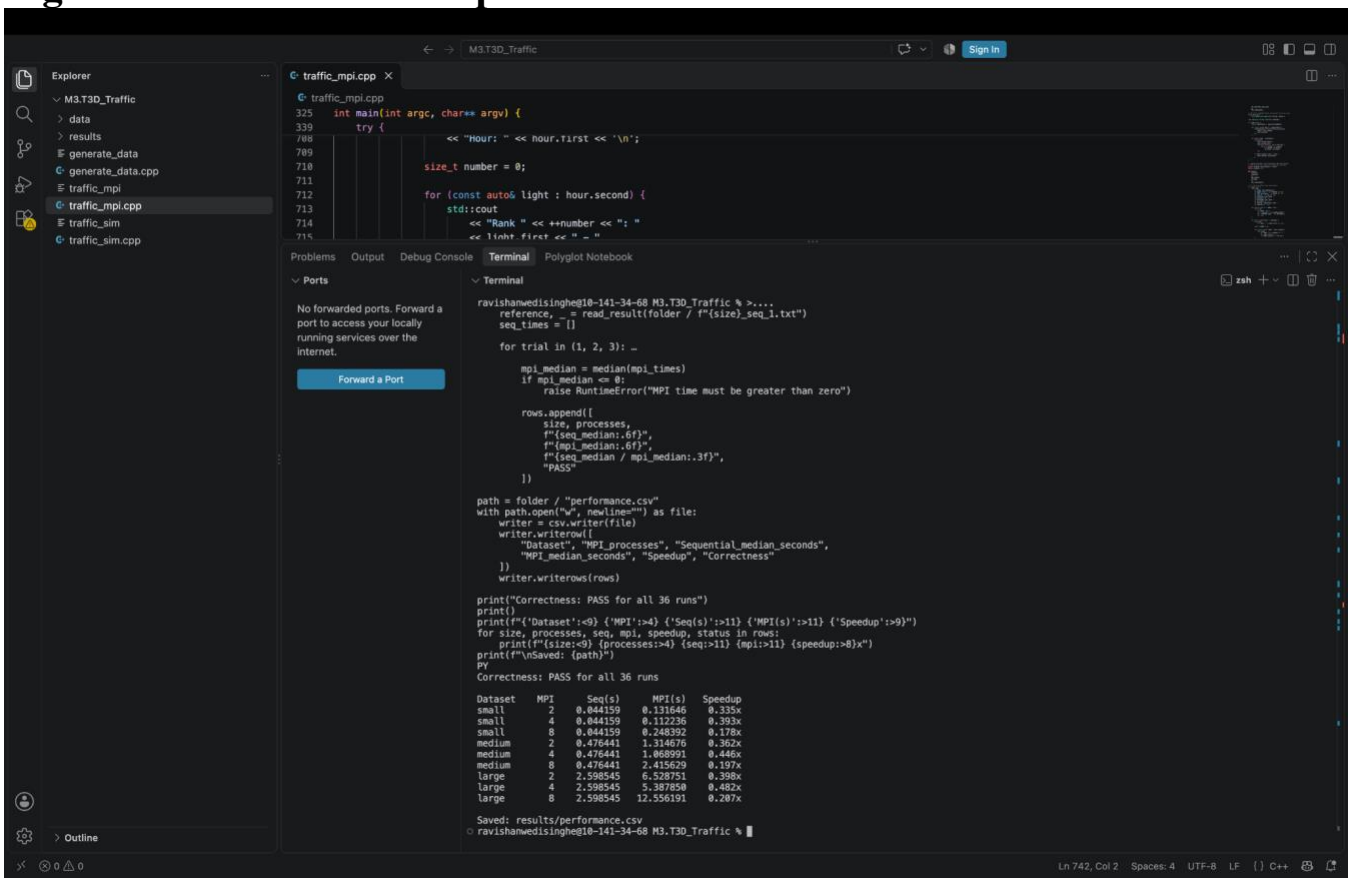
Figure 1 Hand check output



The seven record test gives the expected totals and tie order. It uses two MPI processes, two producers, two consumers and a queue capacity of two.

Original screenshot included in full without cropping. Detailed values and interpretation are given in the report text.

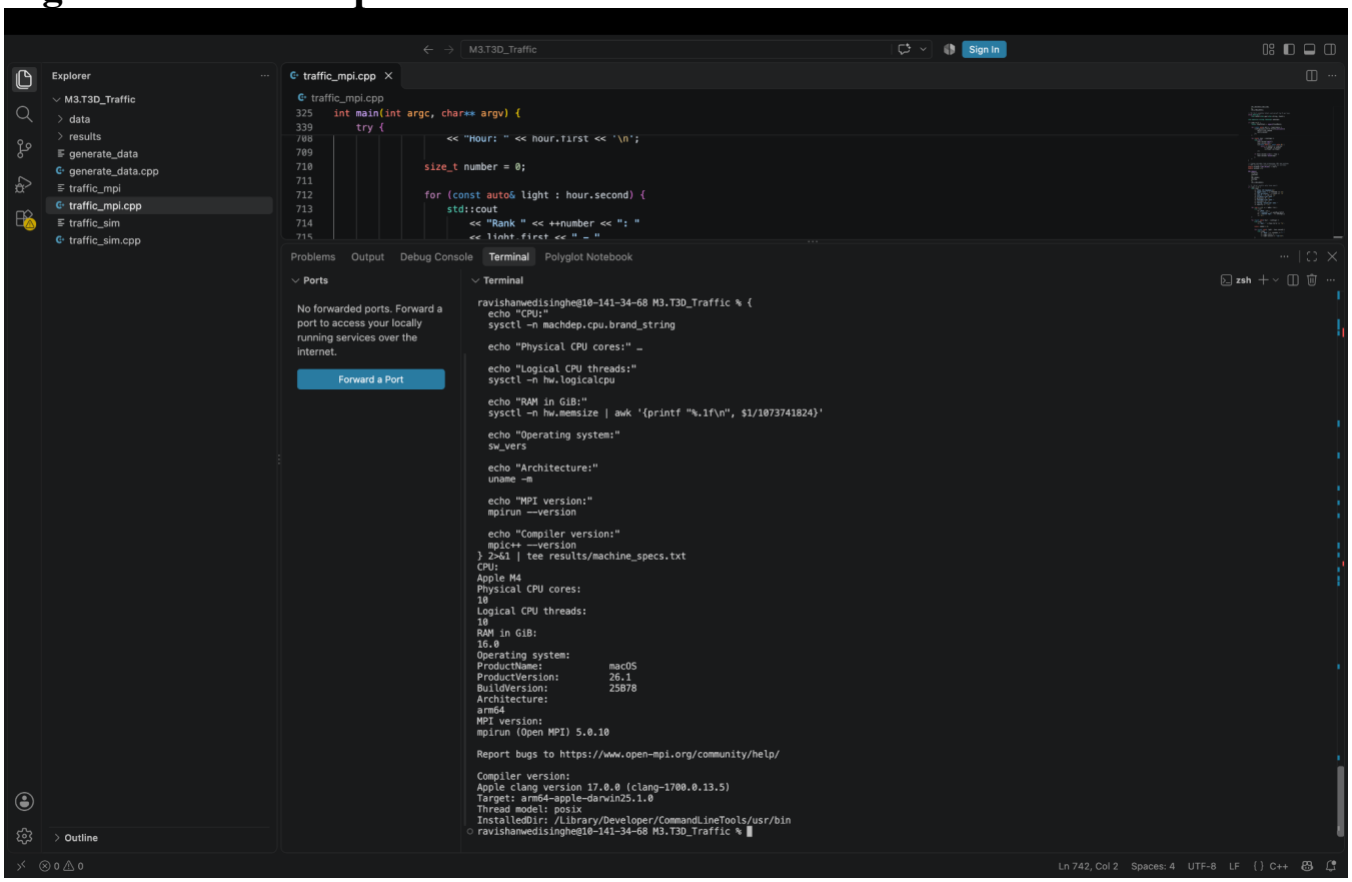
Figure 2 Correctness and performance



All 36 benchmark runs passed the reported output comparison. The table shows median processing times and speedup for three workloads and three MPI process counts.

Original screenshot included in full without cropping. Detailed values and interpretation are given in the report text.

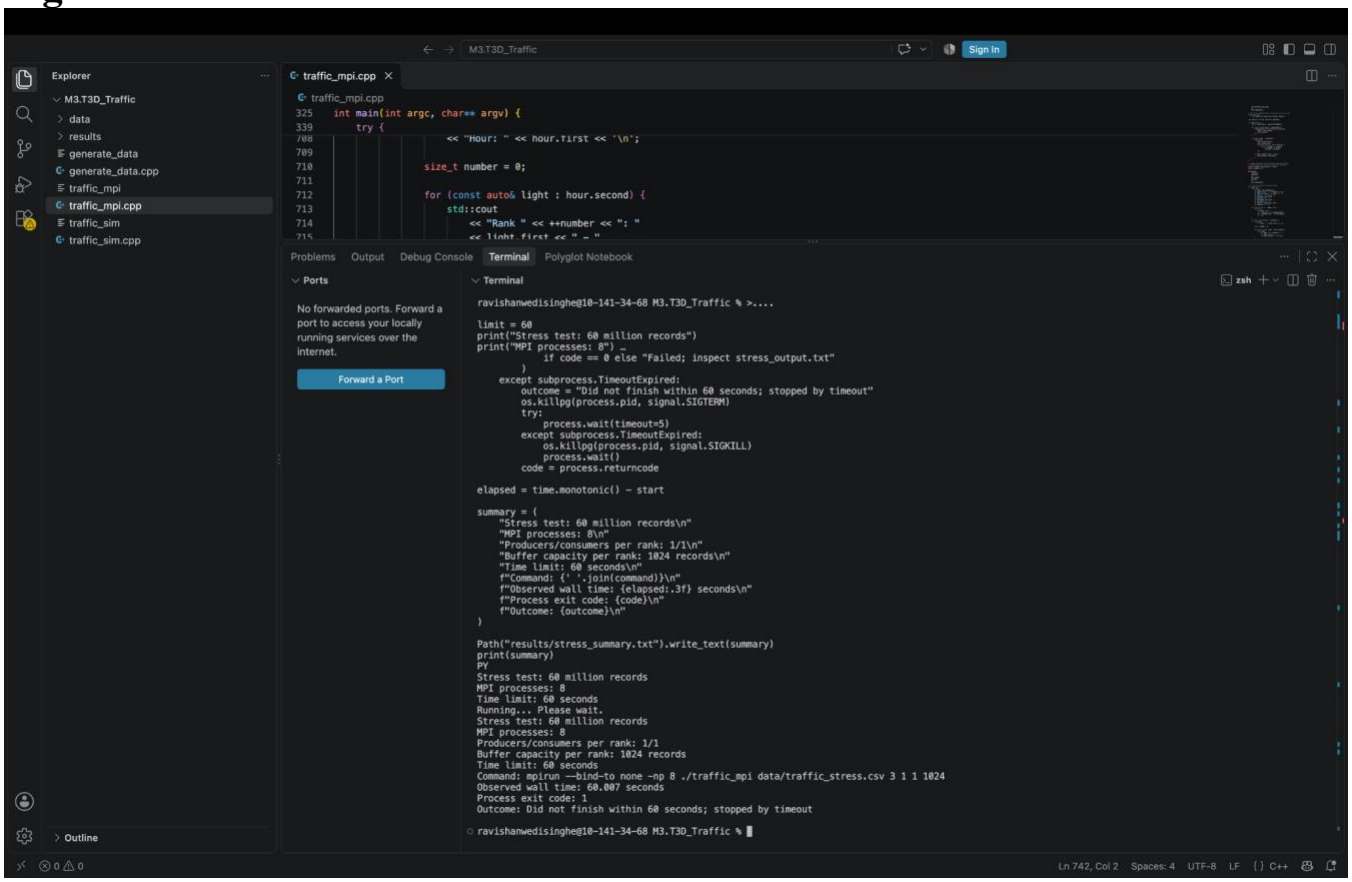
Figure 3 Machine specifications



The commands identify the Apple M4 CPU, 10 cores and logical threads, 16 GiB RAM, macOS version, MPI version and compiler.

Original screenshot included in full without cropping. Detailed values and interpretation are given in the report text.

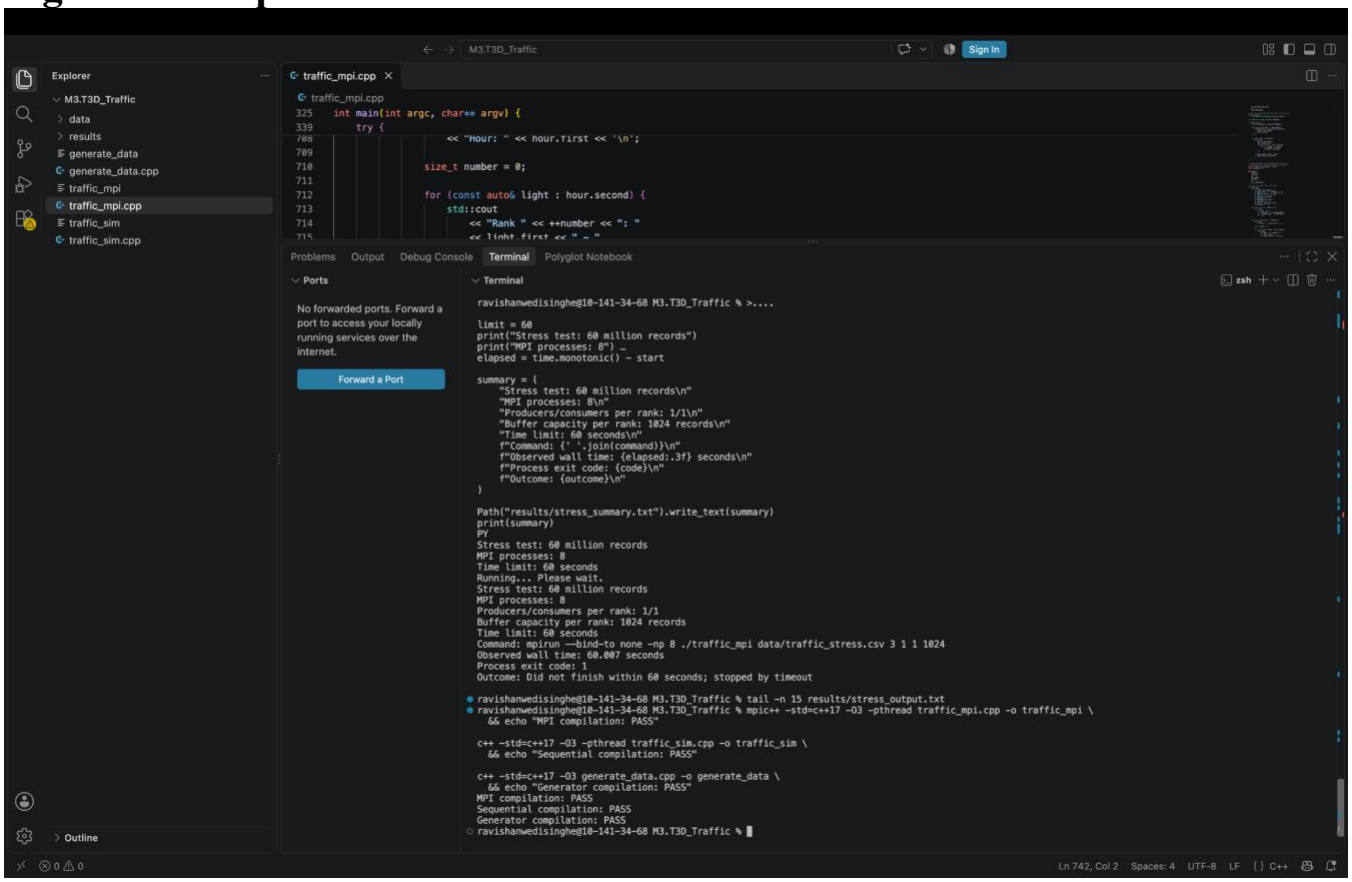
Figure 4 Stress test outcome



The 60 million record run was stopped by the chosen timeout after 60.007 seconds. The launcher exit code was 1 after termination.

Original screenshot included in full without cropping. Detailed values and interpretation are given in the report text.

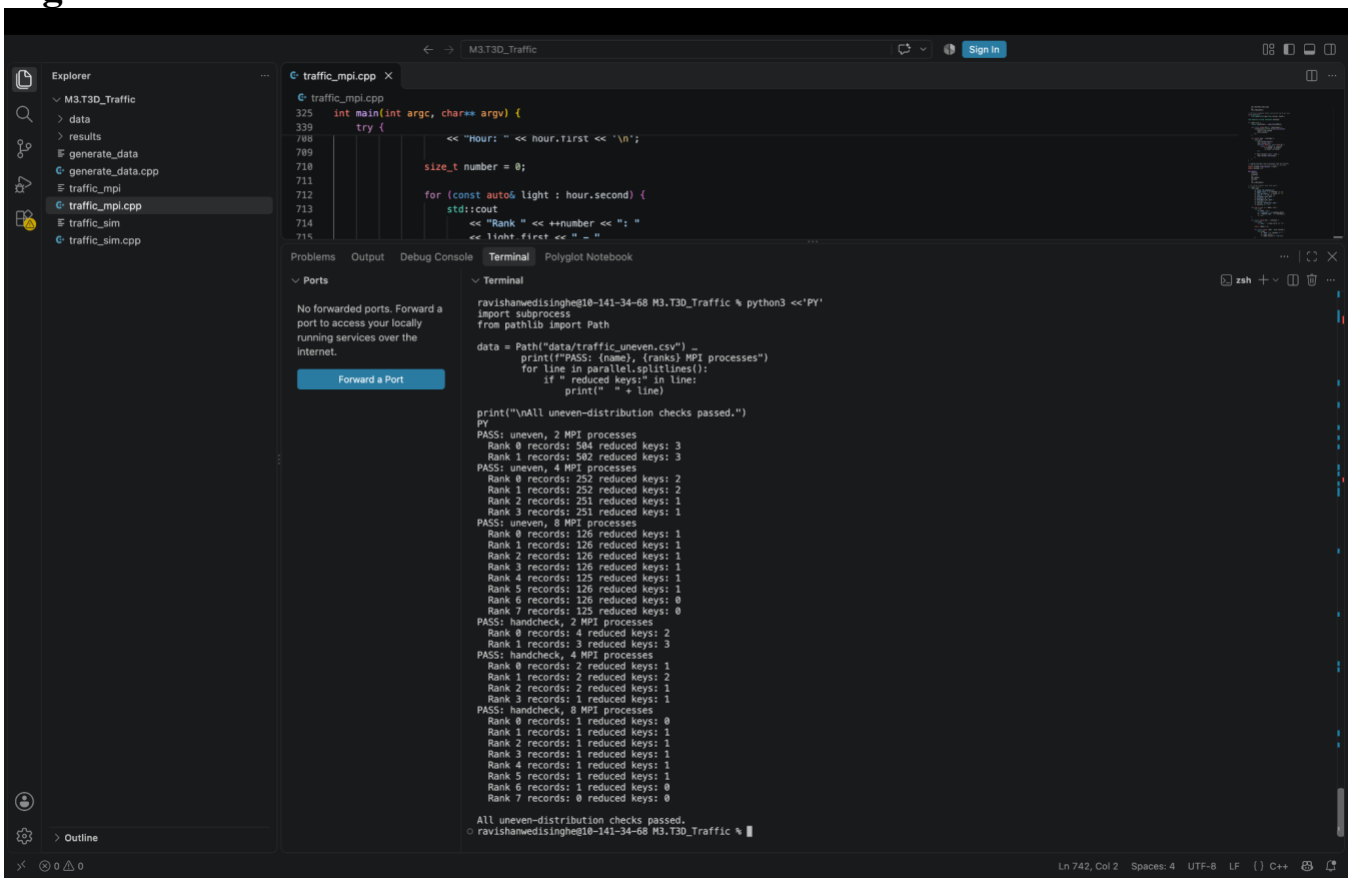
Figure 5 Compilation evidence



The MPI program, sequential program and data generator all compiled successfully. The stress log check printed no messages.

Original screenshot included in full without cropping. Detailed values and interpretation are given in the report text.

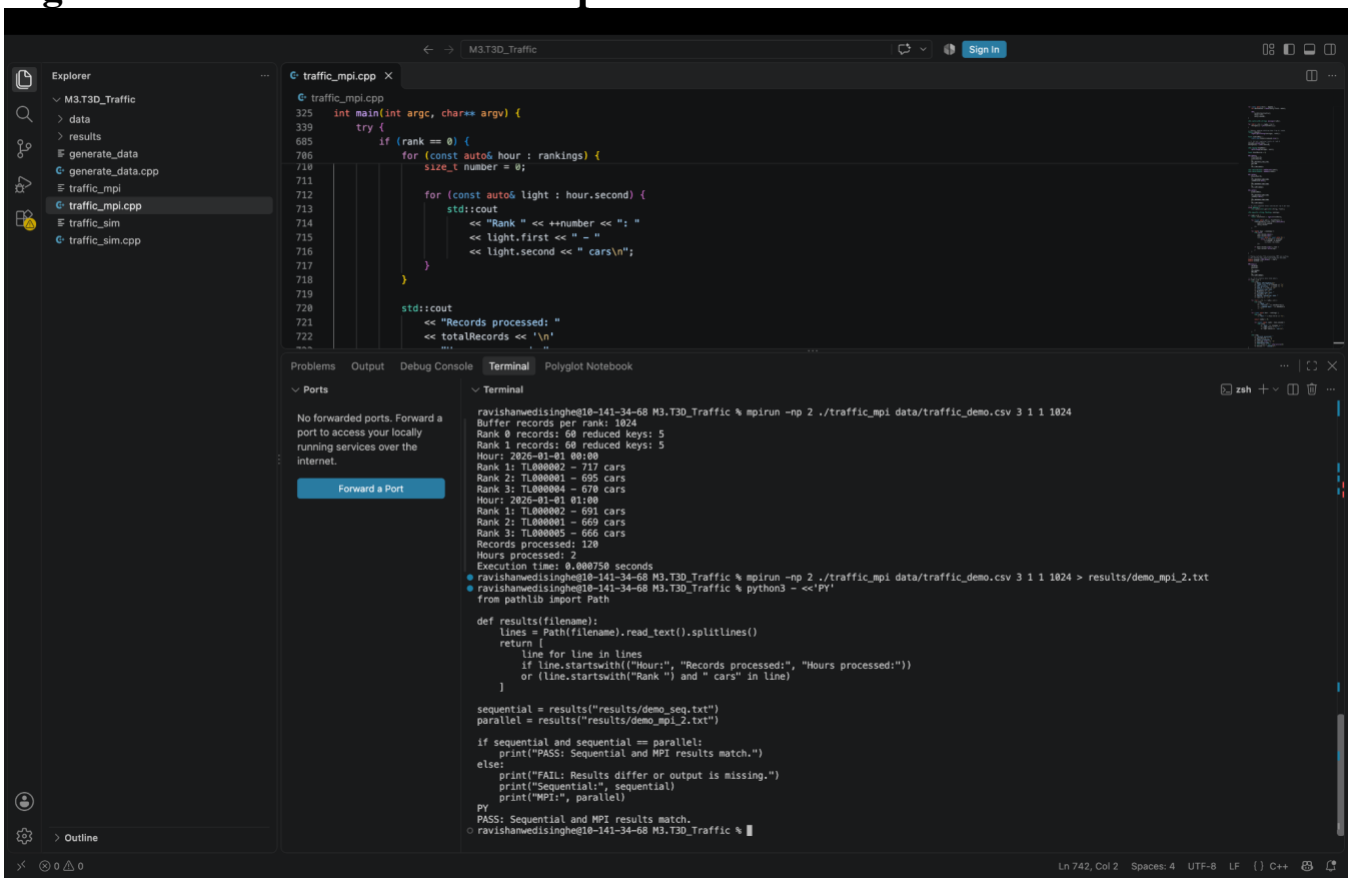
Figure 6 Uneven distribution evidence



Both datasets passed at 2, 4 and 8 processes. The output includes ranks with no reduced keys and a rank with no input records.

Original screenshot included in full without cropping. Detailed values and interpretation are given in the report text.

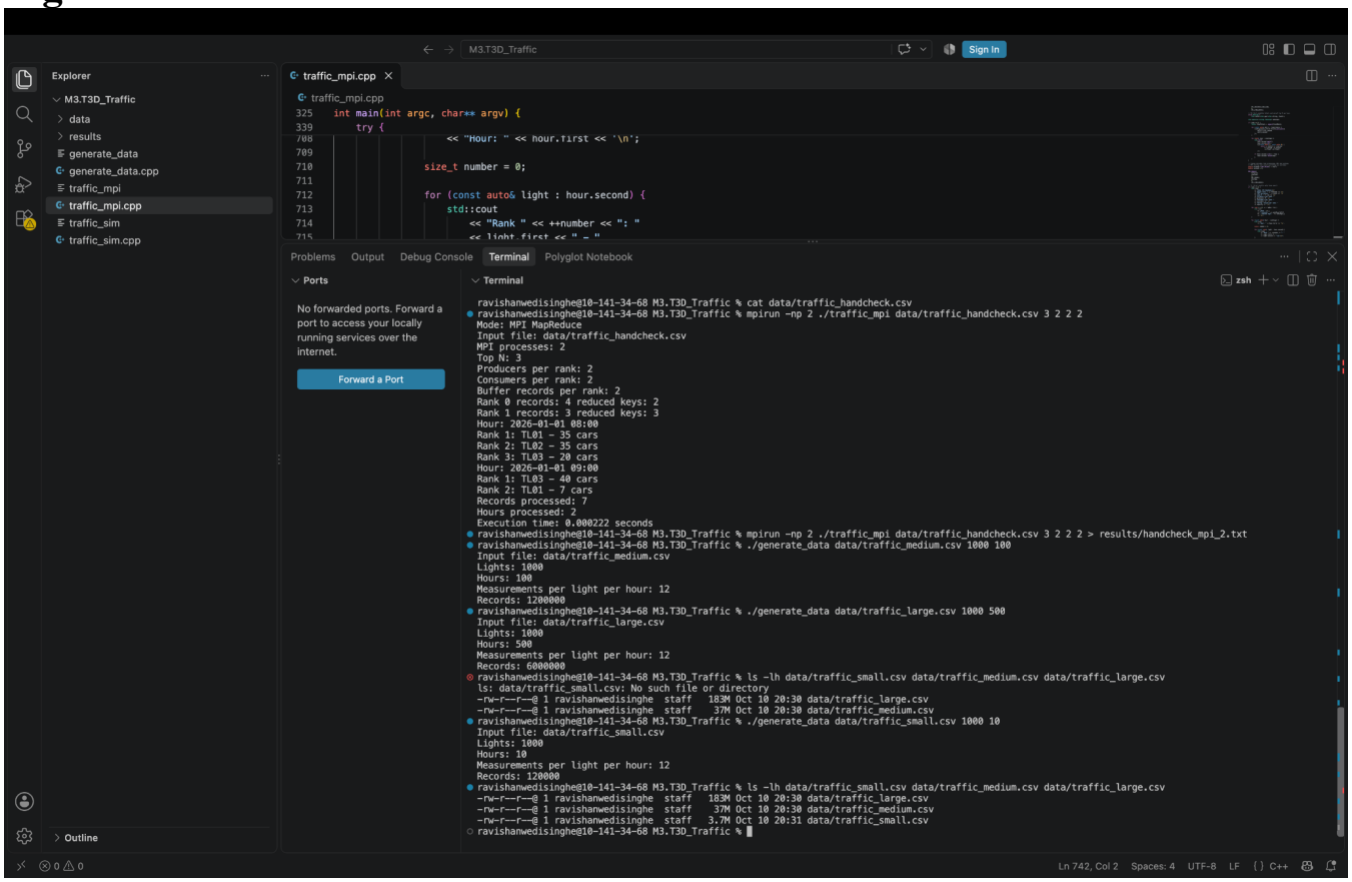
Figure 7 Demo correctness comparison



The demo processes 120 records across two hours. Each of the two ranks reads 60 records. The comparison script reports that the sequential and MPI hourly rankings and processed counts match.

Original screenshot included in full without cropping.

Figure 8 Creation of benchmark datasets



The generator creates 120000 small records, 1200000 medium records and 6000000 large records. The final file listing confirms that all three files exist. The earlier missing small file message was resolved by generating that file.

Original screenshot included in full without cropping.

video demonstration - https://deakin.au.panopto.com/Panopto/Pages/Viewer.aspx?id=2fe4b382-e729-405a-9d12-b4e000757251